

gemstone-py Performance

Benchmark methodology, current baseline numbers, and regression policy

Generated from the Markdown sources in gemstone-py/docs
Assets are local SVG illustrations stored in the repository.

Contents

Performance

[Current Committed Baseline](#)

[Run The Benchmarks](#)

[Round-Trip Reduction APIs](#)

[Native Backend Microbenchmark](#)

[CI Policy](#)

Performance

`gemstone-py` keeps performance claims tied to benchmark artifacts. The repository ships a maintained benchmark CLI, committed baselines, and a comparison workflow so performance regressions can be reviewed as data instead of anecdotes.

Current Committed Baseline

The current public baseline is `.github/benchmarks/baseline.json`. It was generated on 2026-04-20 with:

Field	Value
Platform	<code>macOS-26.3.1-arm64-arm-64bit-Mach-0</code>
Python	<code>CPython 3.14.3</code>
Stone	<code>gs64stone</code> on <code>localhost</code>
Entries	<code>200</code>
Search runs	<code>10</code>
Suites	<code>persistent_root</code> , <code>gscollection</code> , <code>gstore</code> , <code>rchash</code>

These numbers are a small smoke-sized profile, not a universal capacity claim. They are useful as a reference point and as a regression baseline for the configured self-hosted GemStone runner.

Suite	Operation	Count	Elapsed	Approx. per operation	Ops/s	Notes
<code>persistent_root</code>	<code>write_mapping_commit</code>	200	0.1099s	0.550ms	1,819	Writes Python mapping commit
<code>persistent_root</code>	<code>mapping_keys</code>	200	0.0006s	0.003ms	337,316	Reads commit mapping
<code>gscollection</code>	<code>bulk_insert_and_index_commit</code>	200	0.0980s	0.490ms	2,040	

Suite	Operation	Count	Elapsed	Approx. per operation	Ops/s	Notes
						Bulk record creation index
gscollection	indexed_search	10	0.0126s	1.263ms	792	materialize
gstore	batch_write	200	0.0929s	0.464ms	2,153	Transaction key, write
gstore	snapshot_read	200	0.0689s	0.344ms	2,903	Transaction key, read
rhash	populate_commit	200	0.0159s	0.079ms	12,581	Populate RCH commit
rhash	items	200	0.0013s	0.006ms	154,644	Read commit hash

Run The Benchmarks

Run the default live benchmark suite against your configured stone:

```
./scripts/run_benchmarks.sh
gemstone-benchmarks --entries 500 --search-runs 20
```

Capture a JSON artifact:

```
./scripts/run_benchmarks.sh --json --output benchmark-report.json
```

The `gscollection` suite records both the legacy materialized path and the streaming path:

- `indexed_search` and `indexed_search_iter` compare list-returning indexed search with chunked indexed iteration.
- `all_materialize` records `collection.all()` latency and peak Python allocation.

- `iter_stream_count` records the same full-collection pass while streaming through `collection.iter()`.

Use a larger entry count, such as `--entries 50000`, when you want to compare large-result memory behavior rather than smoke-test latency.

Round-Trip Reduction APIs

The lightweight performance direction is to reduce avoidable GCI round trips without adding a client-side identity map. The main helpers are:

- `PersistentRoot.get_many(...)` and `PersistentRoot.update_many(...)` for top-level `UserGlobals` batches.
- `GsDict.get_many(...)` and `GsDict.update_many(...)` for nested `StringKeyValueDictionary` batches.
- `GemStoneSession.bulk_perform_value(...)` and `bulk_perform_oop(...)` for one selector across many receivers.
- `GemStoneSession.bulk_perform_calls_value(...)` and `bulk_perform_calls_oop(...)` for mixed receiver/selector calls.
- `perform_many_value(...)`, `perform_many_oop(...)`, `perform_calls_value(...)`, and `perform_calls_oop(...)` as readability aliases.

Use these when the unit of work is already a batch. They are not a substitute for good repository-side indexing or a reason to pull large object graphs into Python.

Compare two saved reports:

```
gemstone-benchmark-compare baseline.json candidate.json
gemstone-benchmark-compare baseline.json candidate.json --json --output benchmark-compare.json
```

Select the committed baseline that matches the generated report metadata:

```
python -m gemstone_py.benchmark_baselines benchmark-report.json \
--manifest .github/benchmarks/index.json
```

Register a new accepted baseline:

```
gemstone-benchmark-baseline-register benchmark-report.json
```

Native Backend Microbenchmark

The `gci` suite compares low-level helper-call overhead for the pure `ctypes` backend and the optional PyO3 native backend. It does not require a live stone:

```
gemstone-benchmarks --suite gci --entries 1000000
```

To compare real GemStone workloads through each backend, run the same live benchmark twice with an explicit backend and compare the artifacts:

```
GEMSTONE_PY_GCI_BACKEND=ctypes gemstone-benchmarks --json --output ctypes-report.json  
GEMSTONE_PY_GCI_BACKEND=native gemstone-benchmarks --json --output native-report.json  
gemstone-benchmark-compare ctypes-report.json native-report.json
```

CI Policy

The manual `Benchmarks` workflow runs on the GemStone-capable self-hosted runner, uploads the fresh benchmark report, selects the best committed baseline, and enforces configured regression thresholds with `gemstone-benchmark-compare`.

The committed thresholds are deliberately per-operation rather than one global number. Write-heavy operations and indexed search have different run-to-run jitter, so a single global threshold either hides meaningful regressions or fails on normal noise.

When updating these numbers, commit both the new baseline report and the manifest update under `.github/benchmarks/`.

This PDF was generated locally from the Markdown sources in `docs/`. If you edit the source files, rerun `python docs/build_pdf_docs.py`.